Formal Symbols in $\mathcal{UD}$

Brian Beckman
Feb 2026

In many tensorial calculations in $\mathcal{UD}$, we see formal symbols like $\mathbf{z}$ or extended formal symbols like $\mathbf{m123}$. Formal symbols are like ordinary symbols, except they're protected: you can't assign values to them without unprotecting them. Mathematica supports only "Pure System" formal symbols, that is, formal symbols consisting of exactly one character. We need extended formal symbols, with some characters after the underdotted lead character, because we need to generate symbols that are guaranteed not to collide with other symbols, and because we sometimes need generated symbols in contexts where formal symbols are best.

Wolfram does not offer FormalSymbolQ, so we implement it here. Extended formal symbols are an invention: Wolfram knows nothing about them, so we implement a constructor, CreateExtendedFormal, and a predicate, FormalSymbolExtendedQ. Any symbol can be a formal symbol, an extended formal symbol, or neither, but never both.

We further implement a SymbolInspector for human monitoring and debugging of formal symbols.

This exercise blossomed into a self-taught master class in evaluation control. The last section of this notebook presents our learning.

In[1]:=
```
With[{dir = "/Users/brian/Dropbox/Mac/Documents/GitHub/RelativityToolkit/"},
  << (dir <> "RelativityToolkit.wl");
  << (dir <> "RelativityToolkit_RegressionTests.wl");];
```

» Relativity Toolkit version : 1.10.0

» Relativity Connection Symbol : Γ

```
================================================================
RUNNING REGRESSION SUITE v1.10.0
================================================================

--- SECTION 1: TYPE CHECKING (VALENCE) ---
[PASS] Vector (x^μ) ⟶ {{μ}, {}}
[PASS] Covector (p_ν) ⟶ {{}, {ν}}
[PASS] Mixed Tensor (T^μ_ν) ⟶ {{μ}, {ν}}
[PASS] Mixed Tensor (T_μν) ⟶ {{}, {μ, ν}}
[PASS] Mixed Tensor (T^μν) ⟶ {{μ, ν}, {}}
[PASS] Mixed Tensor (T^ν_μ) ⟶ {{ν}, {μ}}
[PASS] Bare Symbol T ⟶ {{}, {}}
```

[PASS] Bare Call T[] ⟶ {{}, {}}

[PASS] Trap (Null) ⟶ {{}, {}}

[PASS] Scalar Contraction (x^{μ} p_{μ}) ⟶ {{}, {}}

[PASS] Scalar Contraction (x^{μ} p_{μ}) ⟶ {{}, {}}

[PASS] Tensor-Vector (x^{ν} $T^{\mu}{}_{\nu}$ → x^{μ}) ⟶ {{μ}, {}}

[PASS] Tensor-Vector (x^{μ} $T^{\mu}{}_{\nu}$ → x_{ν}) ⟶ {{}, {ν}}

[PASS] Power of Scalar ⟶ {{}, {}}

[PASS] Scalar Multiplication ⟶ {{μ}, {}}

[PASS] Gradient d(f)/dx^{α} -> Down[α] ⟶ {{}, {α}}

Valence Mismatch

» expect error message above {{}, {}}

[PASS] Mismatch Handled Gracefully ⟶ {{}, {}}

--- SECTION 2: ALGEBRAIC CANONICALIZATION ---

[PASS] Dummy Renaming (A^{μ} B_{μ} ==A^{ν} B_{ν}) ⟶ $\left(p_{i1}\right)\left(x^{i1}\right)$

[PASS] Commutativity (A^{μ} B_{μ} == B_{ν} A^{ν}) ⟶ $\left(p_{i1}\right)\left(x^{i1}\right)$

[PASS] Index Coalescing (A^{μ} B_{μ} + A^{ν} B_{ν} == 2 A^{ρ} B_{ρ}) ⟶ $2\left(p_{i1}\right)\left(x^{i1}\right)$

[PASS] Double Dummy Pairs (P^{μ} S^{ν} Q_{μ} T_{ν}) ⟶ $\left(P^{i1}\right)\left(Q_{i1}\right)\left(S^{i2}\right)\left(T_{i2}\right)$

[PASS] Free Indices Preserved ⟶ $\left\{\left(A^{\alpha}\right)\left(B^{\beta}\right),\left(A^{i1}\right)\left(B^{i1}\right)\right\}$

--- SECTION 3: PHYSICS, METRIC & TRANSFORMATIONS ---

[PASS] Lowering Index (A^{ν} $g_{\mu\nu}$ → A_{μ}) ⟶ A_{μ}

[PASS] Raising Index ($g^{\mu\nu}$ p_{ν} → p^{μ}) ⟶ p^{μ}

[PASS] Inverse Metric ($g^{\mu\alpha}$ $g_{\alpha\nu}$ → δ^{μ}_{ν}) ⟶ δ^{μ}_{ν}

[PASS] Alpha-Conversion: distinct indices) ⟶ {μ11, μ12}

[PASS] Triple Metric Sandwich (g.g.g -> g) ⟶ $g_{\mu\ \nu}$

[PASS] Delta-Gamma Contraction ⟶ $\Gamma^{s}_{a\,b}$

=== Metric Differentiation Rules ===

[PASS] Permissive Contraction ⟶ δ^{nu}_{lam}

[PASS] Metric Symmetry in Derivatives ⟶ 0

[PASS] Delta Chain Rule ⟶ $f[x]_{,\mu}$

[PASS] Metric Compatibility ⟶ 0

[PASS] Levi-Civita Index Safety (Unique indices generated) ⟶ {σ17, σ18}

--- SECTION 4: METRIC EQUIVALENCE (A^{μ} B_{μ} == B^{ν} A_{ν}) ---

[PASS] Structural Distinction (A^{μ} B_{μ} ≠ B^{ν} A_{ν}) initially ⟶ $\left\{\left(A^{i1}\right)\left(B_{i1}\right),\left(A_{i1}\right)\left(B^{i1}\right)\right\}$

[PASS] Metric Equivalence (A_ν B^ν reduces to A^μ B_μ) ⟶ $\left(A^{i1}\right)\left(B_{i1}\right)$

--- SECTION 5: SECOND DERIVATIVE Box Structure ---

[PASS] Found FractionBox + ^2 ⟶ True

--- SECTION 6: CALCULUS ENGINE ---

[PASS] Leibniz Product Rule ⟶ $g\, f_{,\mu} + f\, g_{,\mu}$

[PASS] Covariant Derivative Expansion ⟶ True

[PASS] Power Rule ⟶ $2\left(a\$7575^{i1}\right)\left(a\$7575^{i1}{}_{,\nu}\right)$

[PASS] Differentiation Linearity ⟶ True

[PASS] CD of Scalar reduces to Partial ⟶ $phi\$7588_{,\mu}$

[PASS] Scalar CD contains no Connection Coefficients ⟶ True

[PASS] Nested CD Alpha-Conversion (Unique Indices Generated) ⟶ True

--- SECTION 7: QUOTIENT THEOREM (Extraction) ---

[PASS] Quotient Extraction (Simple) ⟶ T_{s}

[PASS] Quotient Extraction (Distributed Dummies) ⟶ $R_{s} + S_{s}$

[PASS] Quotient Extraction (Zero Case) ⟶ 0

--- SECTION 8: TENSOR FORM ROBUSTNESS ---

[PASS] TensorForm maps arbitrary formals (Q, Z, I99) to Greek ⟶ True

--- SECTION 9: COMMA NOTATION DISPLAY LOGIC ---

[PASS] Atomic Tensor ⟶ $A^{\mu}{}_{,\nu}$

[PASS] Sealed Product (Gamma Glue) ⟶ $((A^{\mu})\,\Gamma^{a}_{bc})_{,\nu}$

[PASS] Sealed Sum ⟶ $(A + B)_{,\nu}$

[PASS] Sealed Power ⟶ $\left(A^2\right)_{,\nu}$

[PASS] Arbitrary Function (Sin) ⟶ $Sin[theta]_{,\nu}$

[PASS] Nested Math (A*(B+C)) ⟶ $(A\,(B + C))_{,\nu}$

--- SECTION 10: EINSTEIN SUMMATION (Contract & ContractAll) ---

[PASS] Single Contraction (A^u B_u) ⟶ $2\left(A^{i1}\right)\left(B_{i1}\right)$

[PASS] Contract (One-Shot) expanded exactly one pair ⟶ {False, True}

[PASS] ContractAll expanded all pairs recursively ⟶ True

[PASS] Variance Handling (Divergence: d(A^u)/dx^u) ⟶ $2\left(A^{i1}{}_{,i1}\right)$

[PASS] Distribution over Sum (A^u B_u + C^v D_v) ⟶ True

[PASS] Free Indices Protected (No Contraction) ⟶ $(A^{\mu})\,(B_{\nu})$

[PASS] Coordinates Excluded from Scanner ⟶ A^{1}

```
--- SECTION 11: ELECTRODYNAMICS & GAUGE INVARIANCE ---
[PASS] U(1) Gauge Covariance ((ψ D_μ)'==e^(ieΛ) ψ D_μ) ⟶ -i e^(i e$7688 Λ$7688) e$7688 ψ$7688 (A$7688 _μ) + e^(i e$7688 Λ$7688) ψ$7688,_μ
[PASS] Field Strength Invariance (F' == F) ⟶ -(A$7688 _m,n) + A$7688 _n,m
[PASS] Weinberg-Salam Mixed Covariance (Geometric + Gauge) ⟶ e^(i gW$7703 Λ$7703) (W$7703 ^α,_μ) - i e^(i gW$7703 Λ$7703) gW$7703 (W$7703 ^α) (Z$7703 _μ) + e^(i gW$7703 Λ$7703) (W$7703 ^j1) Γ^α_μ j1

--- SECTION 12: RIEMANN CURVATURE DERIVATION ---
[PASS] Riemann Derivation matches Textbook ⟶ Γ^μ_s ν,ρ - Γ^μ_s ρ,ν - Γ^j1_s ρ Γ^μ_j1 ν + Γ^j1_s ν Γ^μ_j1 ρ

--- SECTION 13: GEOMETRY OPERATORS (Gradient, Laplacian) ---
[PASS] Gamma Factory (Spherical Γ_rθθ) ⟶ -r
[PASS] GradRaised (Radial Function r^2) ⟶ {0, 2 r, 0, 0}
[PASS] GradSquared (Null Vector Norm) ⟶ 0
[PASS] ScalarLaplacian (Harmonic 1/r) ⟶ 0
[PASS] ScalarLaplacian (Polynomial r^2) ⟶ 6

----------------------------------------------------------------------
REGRESSION SUITE COMPLETE
PASSED: 69
FAILED: 0
STATUS: GREEN (Ready for Publication)
----------------------------------------------------------------------
```

Formal Symbol Character Ranges

showCodes generates a visual map of the specific Unicode Private Use Area (63488–63615) that Mathematica uses for formal symbols.

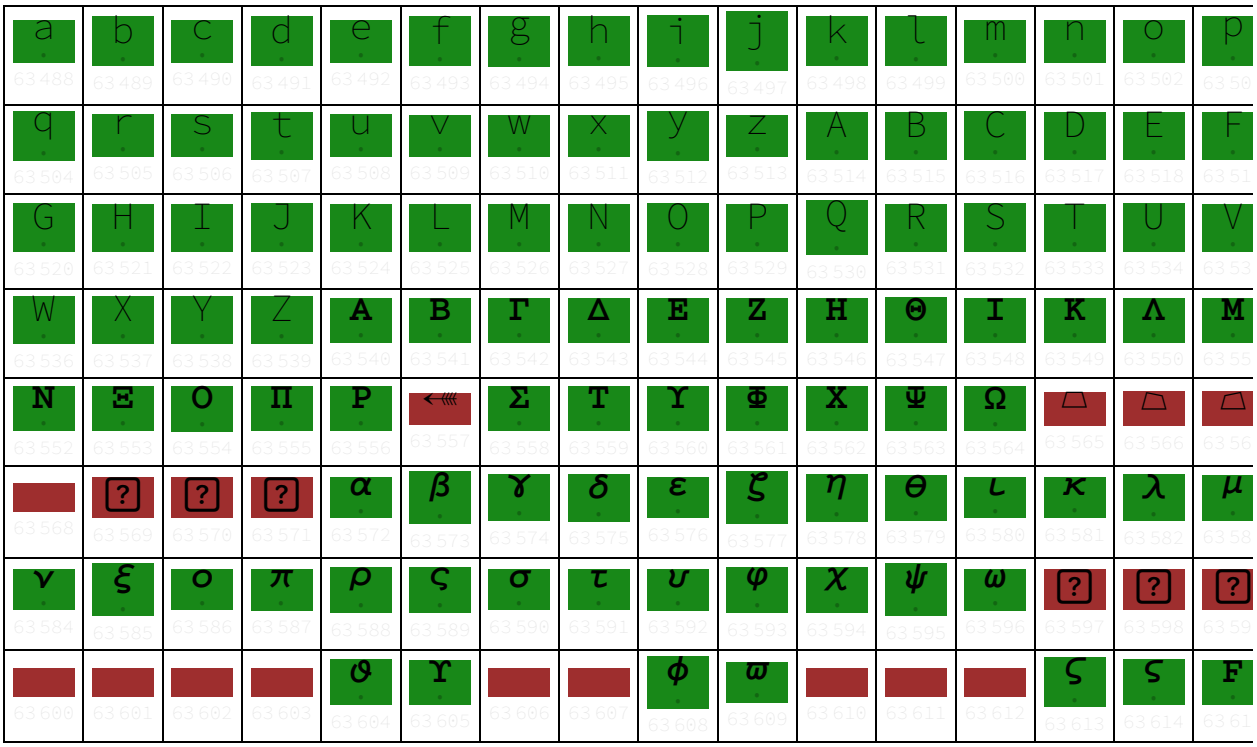
- Green Tiles: Valid, assigned formal symbols (e.g., $\underset{\cdot}{a}$, $\underset{\cdot}{\alpha}$).
- Red Tiles: Unassigned or invalid slots (gaps in the sequence).

The visualization iterates through the integer range and queries **formalCharCodeQ** for every number to decide whether it should be colored Green (Valid) or Red (Invalid).

In[2]:=
```
ClearAll[formalCharCodeQ];
formalCharCodeQ[code_Integer] := (
    (63 488 ≤ code ≤ 63 556) || (*Latin and primary math*)
     (63 558 ≤ code ≤ 63 564) || (*Remaining math symbols after arrow*)
     (63 572 ≤ code ≤ 63 596) || (*Lowercase Greek*)
     (63 604 ≤ code ≤ 63 605) || (*Mathematical variants*)
     (63 608 ≤ code ≤ 63 609) || (*Greek variants*)
     (63 613 ≤ code ≤ 63 615)  (*Final Greek variants*));
```

In[4]:=
```
ClearAll[showCodes];
showCodes[] := Module[{codes = Range[63 488, 63 615]},
    Grid[Partition[MapIndexed[Column[{
            Item[Style[Symbol@FromCharacterCode@#1, 20, Bold],
             Background → If[formalCharCodeQ[#1],
               Darker[StandardGreen],
               Darker[StandardRed]]],
            Style[#2〚1〛 + 63 487, 10, GrayLevel[0.90]]},
           Alignment → Center] &, codes], 16],
     Frame → All,
     ItemSize → {2.5, 2.5},
     Background → {None, None, {}}]];
showCodes[]
```

Out[6]=



Identity Helper

Purpose

checkFormalIdentity is a low-level, polymorphic interrogator of the structure of an input **Symbol** or **String**. It does not evaluate its input. It returns one of **"Pure"**, **"Extended"**, or **"Neither"**. It fails loudly (with **$Failed**) on any other type of input.

This function has a double-layer defense against evaluation, critical when inspecting symbols that might have values assigned.

1. **SetAttributes[..., HoldFirst]**: stops the argument **s** from evaluating before it enters the function. Without this, calling the function on a variable **x** would pass the value of **x** instead of **x**.
2. **Unevaluated[...]**: Is a second line of defense inside the function body. Even inside a held function, passing **s** to certain other functions can trigger evaluation. Wrapping occurrences of **s** in **Unevaluated**, e.g., **SymbolName[Unevaluated[s]]**, on a symbols **s** guarantees that the called function operates on a symbol itself, not its value. **Unevaluated** has no effect on a **String**.

In[7]:=
```
ClearAll[checkFormalIdentity];
SetAttributes[checkFormalIdentity, HoldFirst];
checkFormalIdentity[s : (_Symbol | _String)] :=
  Module[{name, codes},
   name = Which[
     Head[Unevaluated[s]] === Symbol, SymbolName[Unevaluated[s]],
     (* if input is a literal string, not a symbol *)
     StringQ[Unevaluated[s]], s,
     True, Return[$Failed]];
   codes = ToCharacterCode[name];
   Which[
    (Length[codes] == 1 && formalCharCodeQ[First[codes]]), "Pure",
    (Length[codes] > 0 && formalCharCodeQ[First[codes]]), "Extended",
    True, "Neither"
   ]];
checkFormalIdentity[___] := $Failed;
```

Formal Symbol Predicate

Except for the **HoldAll** attribute, this implementation is obvious, given **checkFormalIdentity**.

```
ClearAll[FormalSymbolQ];
SetAttributes[{FormalSymbolQ}, HoldAll];
FormalSymbolQ[s_] := "Pure" === checkFormalIdentity[s];
```

Extended Formal Symbol Predicate

One must write **FormalSymbolExtendedQ[Evaluate[x]]** to check the value of an expression x, even if

that expression is just a symbol whose value is a constructed extended formal symbol below. Many examples and tests below should make this clear.

In[14]:=
```
ClearAll[FormalSymbolExtendedQ];
SetAttributes[{FormalSymbolExtendedQ}, HoldAll];
FormalSymbolExtendedQ[s_] :=
  checkFormalIdentity[s] === "Extended" && MemberQ[Attributes[s], Protected];
```

Constructor for Extended Formal Symbols

Check for valid structure (one formal character followed by any number [including zero] of non-formal characters.

In[17]:=
```
ClearAll[CreateExtendedFormal];
CreateExtendedFormal::invalidStructure =
  "The name '`1`' is invalid. Extended formals must start with
    exactly one formal character followed by non-formal characters.";
Module[{valid},
  valid[codes_] := Length[codes] > 0 &&
    formalCharCodeQ[First[codes]] &&
    ! AnyTrue[Rest[codes], formalCharCodeQ];
  CreateExtendedFormal[name_String] :=
   If[valid[ToCharacterCode[name]],
    With[{s = Symbol[name]},
     Protect[s]; s],
    (Message[CreateExtendedFormal::invalidStructure, name];
     Return[$Failed])];
  CreateExtendedFormal[sym_Symbol] :=
   CreateExtendedFormal[SymbolName[sym]]; ];
```

Test Suite

In[20]:=
```
(*SETUP*)
Protect[i123];
protectedDummy = CreateExtendedFormal["j888"];
CreateExtendedFormal[i999];

Print["--- (1) Identity & Built-in Protection ---"];

(*Case 0: low-level checks*)
AssertEqual[checkFormalIdentity[i],
  "Pure", "0.1. input: formal symbol literal"];
```

```
AssertEqual[checkFormalIdentity["i"], "Pure", "0.2. input: formal symbol string"];
AssertEqual[checkFormalIdentity[j1234],
  "Extended", "0.3. input: extended symbol literal"];
AssertEqual[checkFormalIdentity["j2345"],
  "Extended", "0.4. input: extended symbol string"];
AssertEqual[checkFormalIdentity[foo],
  "Neither", "0.5. non-formal symbol literal"];
AssertEqual[checkFormalIdentity["foo"], "Neither", "0.6. non-formal string"];
AssertEqual[checkFormalIdentity[""], "Neither", "0.7. non-formal empty string"];
AssertEqual[checkFormalIdentity[], $Failed, "0.8. empty string"];
AssertEqual[checkFormalIdentity[42], $Failed, "0.9. integer"];

(*Case 1: System Protection*)
AssertProtected[i, "1. Identity: Built-in i is Protected"];

(*Case 2: Latin Identification*)
AssertTrue[FormalSymbolQ[i], "2. Identity: Latin Pure Formal recognized"];

(*Case 3: Greek Identification*)
AssertTrue[FormalSymbolQ[α], "3. Identity: Greek Pure Formal recognized"];

Print["--- (2) Pointer & Construction Logic ---"];

(*Case 4: Pointer Resolution (MUST EVALUATE THE POINTER)*)
AssertTrue[FormalSymbolExtendedQ[Evaluate@protectedDummy],
  "4. Pointer: Variable holding symbol recognized"];

(*Case 5: Target is Protected*)
(*the Attributes @@ {sym} idiom defeats HoldAll on Attributes *)
AssertProtected[protectedDummy, "5. Pointer: Target symbol j888 is Protected"];

(*Case 6: Negative Constructor Check*)
AssertMatchQ[Quiet[CreateExtendedFormal[""]], $Failed,
  "6. Constructor: Rejects empty string"];

Print["--- (3) Structural Variety & Boundaries ---"];

(*Case 7: Numbered Dummies (Protected it in Setup)*)
AssertTrue[FormalSymbolExtendedQ[i123],
  "7. Structure: Numbered dummy recognized"];

(*Case 8: Prefixed Symbols*)
```

```
AssertFalse[FormalSymbolExtendedQ[Partialx],
  "8. Structure: Prefixed symbol rejected"];

(*Case 9: Pure System is NOT Extended*)
AssertFalse[FormalSymbolExtendedQ[i], "9. Boundary: Pure is NOT Extended"];

(*Case 10: Extended is NOT Pure System*)
AssertFalse[FormalSymbolQ[i123], "10. Boundary: Extended is NOT Pure"];

Print["--- (4) Scoping Integrity (The 'Hold' Check) ---"];

Block[{x = 10, i = 5, i123 = 7},

  (*Case 11: System Symbol Value-Shielding*)
  AssertTrue[FormalSymbolQ[i],
   "11. Scoping: Built-in identity persists despite value 5"];

  (*Case 12: Extended Symbol (non-System) is not protected *)
  AssertFalse[FormalSymbolExtendedQ[i123],
   "12a. Non-system symbol not protected, thus not extended."];

  Protect[i123];
  AssertTrue[FormalSymbolExtendedQ[i123],
   "12b. Scoping: Extended identity after explicit protection"]
 ];

Print["--- (5) Constructor Validation ---"];

(*Case 13: Reject double formal prefix (e.g.,kj...)*)
AssertMatchQ[Quiet[CreateExtendedFormal["kj888"]],
  $Failed, "13. Safety: Reject double formal prefix"];

(*Case 14: Reject plain string without any formal symbol*)
AssertMatchQ[Quiet[CreateExtendedFormal["plainVar"]],
  $Failed, "14. Safety: Reject plain string"];

(*Case 15: Accept valid single formal prefix*)
AssertTrue[Head[CreateExtendedFormal["k888"]] === Symbol,
  "15. Structure: Accept single formal prefix"];

Print["--- (6) Start-Character Enforcement ---"];
```

```
(*Case 16: Reject'Buried' Formal Symbol (e.g.vari123)*)
AssertFalse[FormalSymbolExtendedQ["vari123"],
  "16. Strictness: Reject buried formal (ExtendedQ -> False)"];

AssertFalse[FormalSymbolQ["vari123"],
  "17. Strictness: Reject buried formal (PureQ -> False)"];

(*Case 18: Reject 'Suffix' Extended Symbol (e.g.xi)*)
AssertFalse[FormalSymbolExtendedQ["xi"],
  "18. Strictness: Reject suffix formal (ExtendedQ -> False)"];

(*Case 18: Reject'Suffix' Pure Symbol (e.g.xi)*)
AssertFalse[FormalSymbolQ["xi"],
  "19. Strictness: Reject suffix formal (PureQ -> False)"];

Print["--- (6) Polymorphism ---"]

(*True: The string "i" describes a valid Pure symbol.*)
AssertTrue[FormalSymbolQ["i"],
  "20. Polymorphism: String \"i\" is a Pure symbol"];

(*True: The string "i" describes a valid Extended symbol.*)
AssertTrue[FormalSymbolExtendedQ["i999"],
  "21. Polymorphism: String \"i999\" is an Extended symbol"];

Print["--- TEST SUITE COMPLETE: 100% COVERAGE ---"];
--- (1) Identity & Built-in Protection ---
[PASS] 0.1. input: formal symbol literal ⟶ Pure
[PASS] 0.2. input: formal symbol string ⟶ Pure
[PASS] 0.3. input: extended symbol literal ⟶ Extended
[PASS] 0.4. input: extended symbol string ⟶ Extended
[PASS] 0.5. non-formal symbol literal ⟶ Neither
[PASS] 0.6. non-formal string ⟶ Neither
[PASS] 0.7. non-formal empty string ⟶ Neither
[PASS] 0.8. empty string ⟶ $Failed
[PASS] 0.9. integer ⟶ $Failed
[PASS] 1. Identity: Built-in i is Protected ⟶ True
[PASS] 2. Identity: Latin Pure Formal recognized ⟶ True
```

```
[PASS] 3. Identity: Greek Pure Formal recognized ⟶ True
--- (2) Pointer & Construction Logic ---
[PASS] 4. Pointer: Variable holding symbol recognized ⟶ True
[PASS] 5. Pointer: Target symbol j888 is Protected ⟶ True
[PASS] 6. Constructor: Rejects empty string ⟶ True
--- (3) Structural Variety & Boundaries ---
[PASS] 7. Structure: Numbered dummy recognized ⟶ True
[PASS] 8. Structure: Prefixed symbol rejected ⟶ True
[PASS] 9. Boundary: Pure is NOT Extended ⟶ True
[PASS] 10. Boundary: Extended is NOT Pure ⟶ True
--- (4) Scoping Integrity (The 'Hold' Check) ---
[PASS] 11. Scoping: Built-in identity persists despite value 5 ⟶ True
[PASS] 12a. Non-system symbol not protected, thus not extended. ⟶ True
[PASS] 12b. Scoping: Extended identity after explicit protection ⟶ True
--- (5) Constructor Validation ---
[PASS] 13. Safety: Reject double formal prefix ⟶ True
[PASS] 14. Safety: Reject plain string ⟶ True
[PASS] 15. Structure: Accept single formal prefix ⟶ True
--- (6) Start-Character Enforcement ---
[PASS] 16. Strictness: Reject buried formal (ExtendedQ -> False) ⟶ True
[PASS] 17. Strictness: Reject buried formal (PureQ -> False) ⟶ True
[PASS] 18. Strictness: Reject suffix formal (ExtendedQ -> False) ⟶ True
[PASS] 19. Strictness: Reject suffix formal (PureQ -> False) ⟶ True
--- (6) Polymorphism ---
[PASS] 20. Polymorphism: String "i" is a Pure symbol ⟶ True
[PASS] 21. Polymorphism: String "i999" is an Extended symbol ⟶ True
--- TEST SUITE COMPLETE: 100% COVERAGE ---
```

Symbol Inspector

Design & internal Logic

Purpose

For the displaying and debugging formal symbols, including our extended formal symbols, **`SymbolInspector`** exposes the internal properties of any symbol, indirecting when necessary when its input evaluates to the formal symbol we want to inspect, metaphorically treating the input as a pointer

to the symbolic value we want.

Usage

```
SymbolInspector[symbolLiteral]
(* or *)
SymbolInspector[Evaluate[expressionWithValue]]
```

- On a Literal: `SymbolInspector[x]` inspects `x`.
- On an Expression: e.g., `SymbolInspector[var]` evaluates the expression `var` first, then inspects the resulting value, which may be the value we want.

Examples:

Inspect a formal symbol input as a literal:

In[•]:=
```
SymbolInspector[i]
```

Out[•]=

Property	Value
Literal Symbol	i
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	System`
Character Codes	{63 496}
Classification	Pure System
Attributes	{Protected}
Definition Type	No Definitions

Indirect a *metaphorical pointer*: Evaluate a symbol to get the formal symbol we *Really* want to inspect

In[•]:=
```
Module[{x = i}, SymbolInspector[x]]
```

Out[•]=

Property	Value
Literal Symbol	x$15190
Own Value	i
Points To	i
Context	System`
Character Codes	{63 496}
Classification	Pure System
Attributes	{Protected}
Definition Type	OwnValue (Variable)

Mechanics

`SymbolInspector` relies on three metaprogramming features:

1. `SetAttributes[..., HoldAll]`

   Problem: Normally, if $x = 5$, typing `f[x]` evaluates to `f[5]`: `x` is evaluated before being "sent" into `f`.

   Solution: `HoldAll` prevents `x` from evaluating, letitng the raw symbol `x` through for deeper inspection.

2. `OwnValues` Introspection

   Problem: In Mathematica, variables normally store definitions (if any) in `OwnValues`, whereas functions (`f[x_] := x`) store definitions in `DownValues`.

Solution: To detect whether `x` is a pointer to another symbol, interrogate `OwnValues[x]`. If we find a rule like `{x :> y}`, we know `x` is a pointer.

3. "Safe Peek" Pattern

   Code: `ReleaseHold[Replace[Hold[s], sym_Symbol :> OwnValues[sym], {1}]]`

   This specific idiom extracts a symbol's `OwnValue` without letting the symbol evaluate.

   3.1. `Hold[s]` freezes the symbol.

   3.2. `Replace` swaps the symbol for its `OwnValues` rule list inside the hold.

   3.3. `ReleaseHold` lets us look at the rule list.

Trial & Error Lesson

Simply calling `OwnValues[s]` inside a held function often evaluates `s` too early. The idiom above was the only way we found to get the definition safely.

"Smart Pointer" Logic

The distinctive feature of this tool is the `isPointer` check.

Logic: `MatchQ[eval, {(_ :> _Symbol)}]`

Behavior:

1. If `x` is defined as `x = y` where `y` is a symbol, the Inspector detects this pattern.
2. It "dereferences" the pointer, switching focus to `y`, ensuring that inspecting a variable holding a formal symbol yields the properties of the formal symbol content, not the properties of the "pointer" variable (the container).

In[60]:=
```
ClearAll[SymbolInspector];
SetAttributes[SymbolInspector, HoldAll];
SymbolInspector[s_Symbol] :=
  Module[{ownVals, isPointer, heldSym},
   (*1. Peek inside the pointer safely*)
   ownVals = OwnValues[Unevaluated[s]];
   isPointer = MatchQ[ownVals, {(_ :> _Symbol)}];
   heldSym = If[isPointer,
     Hold[Evaluate[ownVals[[1, 2]]]],
     Hold[s]];
   (*2. Use 'With' to 'lock in' the symbol we are actually inspecting*)
   Replace[heldSym, Hold[sym_] :>
     With[{
       name = SymbolName[Unevaluated[sym]],
       attr = Attributes[Unevaluated[sym]],
       ctx = Context[Unevaluated[sym]]},
      Grid[{
        {"Property", "Value"},
        {"Literal Symbol", HoldForm[s]},
        {"Own Value", If[ownVals === {}, "None (Undefined)", s]},
        {"Points To", If[isPointer, sym, "N/A (Literal)"]},
        {"Context", ctx},
        {"Character Codes", ToCharacterCode[name]},
        {"Classification", Which[
          (* don't forget to Evaluate! *)
          FormalSymbolQ[Unevaluated[sym]], "Pure System",
          FormalSymbolExtendedQ[Unevaluated[sym]], "Extended",
          True, "User"]},
        {"Attributes", attr},
        {"Definition Type", Which[
          ownVals =!= {}, "OwnValue (Variable)",
          DownValues[Unevaluated[s]] =!= {}, "DownValue (Function)",
          True, "No Definitions"]}},
       Frame -> All,
       Alignment -> Left,
       Background -> {None, {GrayLevel[0.3], Black}}]]];
SymbolInspector[___] := $Failed;
```

Case Analysis

We can pass in

- 2x: naked symbols, naked strings,
- 2x: naked symbols and strings in variables

- 2x: construction expressions for formal symbols from symbols or strings (with Evaluate!),
- 2x: constructed formal symbols and from symbols or string in variables.

Considering both **Pure** (non-extended) and **Extended** symbols, that makes 32 cases, plus a few corner cases. We don't cover all the cases in the examples below, but we do point out some recommended usages and some delicate points.

Naked Symbols and Strings

DO THIS: Note the System namespace; Pure formals are in **System`**. Also note that the symbol is automatically protected by Wolfram and has no **OwnValue**.

In[64]:=
```
SymbolInspector[i]
```

Out[64]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	System`
Character Codes	{63 496}
Classification	Pure System
Attributes	{Protected}
Definition Type	No Definitions

DON'T DO THIS: Expect loud failure on strings.

In[65]:=
```
SymbolInspector[".i"]
```

Out[65]=

```
$Failed
```

You can't assign to a pure system formal, by Wolfram design:

In[66]:=
```
z = 42;
```

Set: Symbol z is Protected.

DON'T DO THIS: A Module will let you have the illusion of assigning a value to a formal symbol, but the symbol won't be any kind of formal. It's a **User** symbol rather than **Pure Formal** or **Extended**, the only alternatives to **User**.

In[67]:=
```
Module[{i = 5}, SymbolInspector[i]]
```

Out[67]=

Property	Value
Own Value	5
Points To	N/A (Literal)
Context	System`
Character Codes	{63 496, 36, 55, 57, 49, 51}
Classification	User
Attributes	{Temporary}
Definition Type	OwnValue (Variable)

DON'T DO THIS: You CAN assign values to things that look like naked extended formals, but it is highly discouraged. They're not extended. They appear in the **User** classification. The only real extended formals are constructed ones (examples below). There is no such thing as a literal extended formal. The constructor will protect constructed extended formals, as we see below, so you can't assign values to them after-the-fact.

In[68]:=
```
zDontDoThis = 42;
SymbolInspector[zDontDoThis]
```

Out[69]=

Property	Value
Own Value	42
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 513, 68, 111, 110, 116, 68, 111, 84, 104, 105, 115}
Classification	User
Attributes	{}
Definition Type	OwnValue (Variable)

DON'T DO THIS: Even without values, things that look like literal extended symbols just are not.

In[70]:=
```
SymbolInspector[z888]
```

Out[70]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 513, 56, 56, 56}
Classification	User
Attributes	{}
Definition Type	No Definitions

Naked Symbols in Variables

Everything above works as expected, just through variables acting, metaphorically, as pointers.

DO THIS:

In[71]:=
```
Module[{var = i}, SymbolInspector[var]]
```

Out[71]=

Property	Value
Own Value	i
Points To	i
Context	System`
Character Codes	{63 496}
Classification	Pure System
Attributes	{Protected}
Definition Type	OwnValue (Variable)

Directly Constructed Symbols

DO THIS: Here is a way to create an extended formal without assigning it to a variable. To inspect it with such direct construction, you must precede the call of **`CreateExtendedFormal`** with an **`Evaluate`**.

In[72]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal[k123]]
```

Out[72]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 498, 49, 50, 51}
Classification	Extended
Attributes	{Protected}
Definition Type	No Definitions

DO THIS: After you do this, the symbol itself, as a literal, now has properties that mark it as a bona-fide extended formal:

In[73]:=
```
SymbolInspector[k123]
```

Out[73]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 498, 49, 50, 51}
Classification	Extended
Attributes	{Protected}
Definition Type	No Definitions

DON'T DO THIS: Recall that if it has not been constructed, it's not an extended formal:

In[74]:=
```
SymbolInspector[k456]
```

Out[74]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 498, 52, 53, 54}
Classification	User
Attributes	{}
Definition Type	No Definitions

DO THIS: If you construct an extended formal from a pure system formal, it's ok. It will (idempotently) stay a pure system formal.

In[75]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal[k]]
```

Out[75]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	System`
Character Codes	{63 498}
Classification	Pure System
Attributes	{Protected}
Definition Type	No Definitions

DO THIS: You can construct extended formals from strings:

In[76]:=
```
SymbolInspector[Evaluate@CreateExtendedFormal["k42"]]
```

Out[76]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 498, 52, 50}
Classification	Extended
Attributes	{Protected}
Definition Type	No Definitions

DON'T DO THIS: After construction, you can't assign to it:

In[77]:=
```
k42 = 42;
```

Set: Symbol k 42 is Protected.

DO THIS: As usual, the extended formal is now defined until you Clear it:

In[78]:=
```
SymbolInspector[k42]
```

Out[78]=

Property	Value
Own Value	None (Undefined)
Points To	N/A (Literal)
Context	Global`
Character Codes	{63 498, 52, 50}
Classification	Extended
Attributes	{Protected}
Definition Type	No Definitions

Constructed Symbols in Variables

Works as expected, from strings or from literals

In[79]:=
```
Module[{var = CreateExtendedFormal["m"]}, SymbolInspector[var]]
```

Out[79]=

Property	Value
Own Value	m
Points To	m
Context	System`
Character Codes	{63 500}
Classification	Pure System
Attributes	{Protected}
Definition Type	OwnValue (Variable)

In[80]:=
```
Module[{var = CreateExtendedFormal[j888]}, SymbolInspector[var]]
```

Out[80]=

Property	Value
Own Value	j888
Points To	j888
Context	Global`
Character Codes	{63497, 56, 56, 56}
Classification	Extended
Attributes	{Protected}
Definition Type	OwnValue (Variable)

On Improper Inputs

We will see $Failed's in some slots, so there's little chance of missing a user error.

In[81]:=
```
Module[{x = CreateExtendedFormal["kj888"]}, SymbolInspector[x]]
```

CreateExtendedFormal: The name 'k j888' is invalid. Extended formals must start with exactly one formal character followed by non–formal characters.

Out[81]=

Property	Value
Own Value	$Failed
Points To	$Failed
Context	System`
Character Codes	{36, 70, 97, 105, 108, 101, 100}
Classification	User
Attributes	{HoldAll, Protected}
Definition Type	OwnValue (Variable)

In[82]:=
```
Module[{x = CreateExtendedFormal["j888"]}, SymbolInspector[x]]
```

CreateExtendedFormal: The name 'j888' is invalid. Extended formals must start with exactly one formal character followed by non–formal characters.

Out[82]=

Property	Value
Own Value	$Failed
Points To	$Failed
Context	System`
Character Codes	{36, 70, 97, 105, 108, 101, 100}
Classification	User
Attributes	{HoldAll, Protected}
Definition Type	OwnValue (Variable)

Master Class in Evaluation Control

Wolfram Mathematica is a term-rewriting system, not a function evaluator like Clojure or JavaScript, let alone a function compiler like C or Java. There are enough similarities between term-rewriting and

function-calling that we often speak of “calling a function” in Mathematica, but actually that is the unusual case, as with explicit **Function** forms or **foo[#]&** syntax. Most of the time, we’re applying rules: replacing the mentioning of **SymbolInspector[x]** with the definition of **SymbolInspector** after replacing its argument pattern **s_Symbol** with **x**.

But wait, we don’t want the value of **x** to go in, we want the symbol **x** itself to go in. Mathematica aggressively tries to evaluate (replace by its definition) every term, every symbol, at every opportunity. That makes it tricky to write a rule like **SymbolInspector** that tries to inspect a symbol. Holding on to a symbol inside **SymbolInspector** is like holding on to a slippery fish that wants to get out of your hands and back into the lake!

```
SetAttributes[SymbolInspector, HoldAll]
```

HoldAll stops evaluation of arguments before replacing **SymbolInspector** with its definition.

Without this, if you wrote **SymbolInspector[x]** and it so happened that **x** has the value **5**, Mathematica would evaluate **5** before **SymbolInspector** even starts: the fish slips out of your hands and back into the lake right away. **SymbolInspector** would receive **5**, fail to match **s_Symbol**, and return **$Failed**.

But with the **HoldAll** (or **HoldFirst**) attribute, Mathematica skips argument-evaluation and binds the variable **s** in the body of **SymbolInspector** to the symbol **x**, not to its value **5**.

```
ownVals = OwnValues[Unevaluated[s]];
```

Pass the wriggling fish **s** to **OwnValues** without triggering evaluation.

Even though **s** is held by **SymbolInspector**, passing **s** to another function exposes **s** to the evaluator again. The fish wriggles whenever you try to move it. If **s** is **x** and **x = 5**, **OwnValues[s]** becomes **OwnValues[5[**. Error.

Why not **OwnValues[Hold[s]]**? **OwnValues** expects a **Symbol** argument, not a **Hold** object.

Unevaluated[s] tells **OwnValues**: “Pretend I passed this wrapped in **Hold**, but treat it as the raw argument.” It evaporates after the rewrite of **OwnValues[s]** to its definition.

The Evaluation Override: `Hold[Evaluate[...]]`

The Job: Perform a calculation inside a metaphorical locked box.

The Scenario: You have **ownVals**, a list of rules. You need to extract the target symbol, e.g., **x**, from that list and wrap it in a **Hold** so it doesn’t evaluate to **5**.

The Mechanics:

Hold[...] normally stops everything.

Evaluate[...] idiomatically forces evaluation inside the **Evaluate** wrapper before the **Hold** freezes the result.

Result: **Hold[Evaluate[ownVals[[1,2]]]]** rewrites to **Hold[x]**.

Without **Evaluate**, you would be holding the code **ownVals[[1,2]]** rather than the symbol **x**.

Trojan Horse Injection: `Replace[heldSym, Hold[sym_] :> With[{...}]]`

The Job: move a symbol, without triggering evaluation, from the locked box (**Hold**) into a working environment (**With**).

The "Gotcha": You have **heldSym**, which is **Hold[x]**. You can't just say **sym = ReleaseHold[heldSym]** because x would instantly evaluate into 5. The fish wriggles out of the box and back into the lake.

The Trick:

- Match the pattern **Hold[sym_]**, binding the name **sym** to the unevaluated content **x** inside the hold.
- :→ (**RuleDelayed**) inserts that raw symbol **x** into the body of the **With** statement.
- Crucially, this injection happens *structurally*, bypassing the evaluator.

Scope Lock: `With[{name = SymbolName[Unevaluated[sym]], ...}, ...]`

The Job: Calculate properties once, safely, and freeze them as constants.

Why: Inside the **With** body, **sym** is the raw symbol **x**. If we used **sym** directly in the grid without **Unevaluated**, it would evaluate to **5**. Another opportunity for the fish to escape.

Strategy: Calculate safe strings (**name**, **ctx**, **attr**) in the **With** header (using **Unevaluated** shields). In the **With** body, every instance of **name** in the **Grid** is replaced with the string "**x**". Strings are safe; they don't evaluate.

The Display Wrapper: `HoldForm[s]`

The Job: "Take a snapshot" of the symbol for display purposes.

Difference from **Hold**: **Hold[s]** evaluates to **Hold[x]**. **HoldForm[s]** evaluates to **x** (visually), but it is an inert form, protected from further evaluation. It is purely cosmetic.

Summary Table

In[83]:=
```
Grid[{{Style["Keyword", Bold], Style["Role", Bold],
    Style["Plain English Translation", Bold]}, {"HoldAll", "Gatekeeper",
    "Don't compute the inputs; let me handle them raw."}, {"Unevaluated", "Shield",
    "Pass this symbol to the function, but don't let it explode into a value."},
   {"Hold", "Cage", "Keep this expression frozen indefinitely."},
   {"Evaluate", "Override",
    "Compute this specific part now, even though you are inside a Cage."},
   {"Replace", "Injector", "Move the contents of the Cage into
      a new piece of code without letting them explode."},
   {"HoldForm", "Photograph", "Show the code as it looks, but don't run it."}},
  Frame → All, Alignment → {{Left, Left, Left}, Top},
  Background → {None, {1 → DarkGray}}, Spacings → {1, 1}]
```

Out[83]=

Keyword	Role	Plain English Translation
HoldAll	Gatekeeper	Don't compute the inputs; let me handle them raw.
Unevaluated	Shield	Pass this symbol to the function, but don't let it explode into a value.
Hold	Cage	Keep this expression frozen indefinitely.
Evaluate	Override	Compute this specific part now, even though you are inside a Cage.
Replace	Injector	Move the contents of the Cage into a new piece of code without letting them explode.
HoldForm	Photograph	Show the code as it looks, but don't run it.